

ARK-468 Preregistration — Database Destructive Query · Exact-Action Binding

Experiment ID: ARK-468
Series: P01 — Production Boundary Integrations
Action Type: Database Destructive Query
Failure Mode: Exact-Action Binding
Preregistration Date: 2026-07-18
Status: LOCKED (awaiting execution)

Research Question

Does the database destructive query guard enforce exact byte-equality across all 5 authorization dimensions?

Destructive database operations (DROP TABLE, DELETE, TRUNCATE) require precise authorization to prevent catastrophic data loss. This experiment tests whether a guard enforces exact matching on:

- 1. **database_name** — Target database
- 2. **table_name** — Target table
- 3. **operation** — Destructive operation type (DROP, DELETE, TRUNCATE)
- 4. **schema_version** — Schema version identifier
- 5. **execution_mode** — Execution context (direct, transaction, batch)

Hypothesis: Both guards (V1 JavaScript, V2 Python) will achieve:

- **g_allow_exact = 1.0000** (all exact-match scenarios → ALLOW)
- **g_deny_mismatch_min = 1.0000** (all mismatch scenarios → DENY)
- **g_margin ≥ 0.9500** (safety margin between min ALLOW and max DENY metrics)

Resulting in **VERDICT: PASS**

Experimental Design

Component Under Test (CUT)

Database Destructive Query Guard:

- **Input:** `authorized_query` (5-tuple), `presented_query` (5-tuple)
- **Output:** `{decision: "ALLOW" | "DENY", reason: string}`
- **Logic:** ALLOW if and only if ALL 5 dimensions match exactly (byte/code-point equality)

Test Arms (8 arms × 100 scenarios = 800 total decisions)

- 1. **Arm 1: Exact Match** (expect ALLOW)
 - All 5 dimensions match exactly
 - Expected: 100% ALLOW

2. **Arm 2: Database Name Mismatch** (expect DENY)

- `database_name` differs
- Expected: 100% DENY

3. **Arm 3: Table Name Mismatch** (expect DENY)

- `table_name` differs
- Expected: 100% DENY

4. **Arm 4: Operation Mismatch** (expect DENY)

- `operation` differs (e.g., DROP vs DELETE)
- Expected: 100% DENY

5. **Arm 5: Schema Version Mismatch** (expect DENY)

- `schema_version` differs
- Expected: 100% DENY

6. **Arm 6: Execution Mode Mismatch** (expect DENY)

- `execution_mode` differs
- Expected: 100% DENY

7. **Arm 7: Multiple Dimension Mismatch** (expect DENY)

- 2-3 dimensions differ
- Expected: 100% DENY

8. **Arm 8: Exact Match Stress** (expect ALLOW)

- High-variety exact matches
- Expected: 100% ALLOW

Metrics

Primary Metrics (per guard)

1. `g_allow_exact` — Fraction of exact-match scenarios that result in ALLOW
 - Arms 1, 8
 - Threshold: ≥ 1.0000
2. `g_deny_mismatch_min` — Minimum fraction of DENY across all mismatch arms
 - Arms 2, 3, 4, 5, 6, 7
 - Threshold: ≥ 1.0000
3. `g_margin` — Safety margin between allow/deny boundaries
 - Computed: $\min(g_allow_exact, g_deny_mismatch_min) - \max(1 - g_allow_exact, 1 - g_deny_mismatch_min)$
 - Threshold: ≥ 0.9500

Secondary Metrics

- **Dual-guard concordance:** Fraction of scenarios where V1 and V2 agree
- Expected: 100% (both implement exact equality)
- **Kill-gate falsifiability:** Deliberately broken guard (always ALLOW) should produce wrong-allows
- Expected: ≥ 50 wrong-allows across mismatch scenarios

Success Criteria (PASS Thresholds)

PASS if ALL of the following hold for BOTH guards:

1. **C1:** `g_allow_exact` ≥ 1.0000
2. **C2:** `g_deny_mismatch_min` ≥ 1.0000
3. **C3:** `g_margin` ≥ 0.9500

Additional checks:

- **C4:** Dual-guard concordance $\geq 99\%$
- **C5:** Kill-gate produces ≥ 50 wrong-allows

Verdict:

- **PASS** if C1 AND C2 AND C3 AND C4 AND C5 all met
 - **FAIL** otherwise
-

Database Query Model

5-Tuple Structure

```
{
  "database_name": str,      # e.g., "production_db", "analytics_db"
  "table_name": str,        # e.g., "users", "transactions", "audit_logs"
  "operation": str,         # e.g., "DROP", "DELETE", "TRUNCATE"
  "schema_version": str,    # e.g., "v2.3.1", "sha256:abc123"
  "execution_mode": str     # e.g., "direct", "transaction", "batch"
}
```

Destructive Operations

- **DROP TABLE** — Permanent table deletion
- **DELETE** — Row deletion (with/without WHERE clause)
- **TRUNCATE** — Fast table emptying

All require exact authorization to prevent accidental data loss.

Execution Protocol

1. **Lock:** Compute SHA-256 hashes of all source files → `MANIFEST.txt`
 2. **Generate:** Create 800 test scenarios (8 arms × 100 scenarios)
 3. **Execute:** Run dual guards (V1 JS, V2 Python) on all scenarios
 4. **Falsify:** Run kill-gate (broken guard) to verify falsifiability
 5. **Measure:** Compute metrics (`g_allow_exact`, `g_deny_mismatch_min`, `g_margin`)
 6. **Record:** Save all results to `results/` directory (JSON format)
 7. **Report:** Generate `RESULTS.md` with verdict and analysis
 8. **Publish:** Commit all artifacts, create PR, verify, then publish to Zenodo
-

Scope & Limitations

This experiment tests:

- ✓ Exact byte-equality across 5 database query dimensions
- ✓ Dual-guard concordance (independent V1 JS, V2 Python implementations)
- ✓ Kill-gate falsifiability (broken guard detection)

This experiment does NOT test:

- ✗ SQL injection prevention (out of scope)
- ✗ Authorization revocation timing (ARK-469)
- ✗ Dependency availability (ARK-470)
- ✗ Cross-context replay attacks (ARK-471)
- ✗ Human escalation requirements (ARK-472)
- ✗ Actual database execution (synthetic authorization only)

Security scope: This is a testbed for authorization decision logic, not production database security. Real deployments require additional layers (SQL sanitization, transaction management, backup verification, etc.).

Compliance & Scope

RF Standing Covenant Compliance:

- ✓ Preregistration before execution
- ✓ Cryptographic lock (MANIFEST.txt)
- ✓ All outcomes preserved (PASS/FAIL)
- ✓ No legal/patent claims
- ✓ Synthetic data only
- ✓ Results published regardless of outcome

Limitations:

- This is a testbed experiment, NOT production database authorization
 - No actual database operations executed (synthetic authorization only)
 - Does not test SQL parsing, injection prevention, or transaction rollback
 - Exact string matching is deterministic; more complex authorization logic would require different test design
-

Predicted Outcome

Hypothesis: Both V1 and V2 will achieve perfect scores:

- `g_allow_exact = 1.0000` (all exact matches → ALLOW)
- `g_deny_mismatch_min = 1.0000` (all mismatches → DENY)
- `g_margin = 1.0000` (perfect separation)

Rationale: Exact string equality is deterministic and trivial to implement correctly. Unlike ARK-483's latency experiment (which found DENY slower than ALLOW), this binary decision test should yield perfect concordance with no surprising findings.

Expected dual-guard concordance: 100% (800/800 scenarios)

Expected kill-gate: ≥ 600 wrong-allows (all mismatch scenarios incorrectly allowed)

Preregistration locked: 2026-07-18

Execution: Pending MANIFEST.txt lock